

Python Tools CLI Cheat Sheet

Purpose: Quick reference for running repository Python tools.

Working directory

NAME

working_directory - Switch to repo root.

SYNOPSIS

```
cd C:\Users\dmona\swerveBringupMotorTest\swerveBringupMotorTest1
Set-Location C:\Users\dmona\swerveBringupMotorTest\swerveBringupMotorTest1
```

DESCRIPTION

Required so relative paths resolve consistently.

EXAMPLES

```
cd C:\Users\dmona\swerveBringupMotorTest\swerveBringupMotorTest1
```

tools.can_nt.can_nt_bridge

NAME

can_nt_bridge - PC-side CAN sniffer, NT publisher, PCAP logger, and optional UI.

SYNOPSIS

Module:

```
python -m tools.can_nt.can_nt_bridge [-h] [--profile PROFILE] [--interface INTERFACE]
[--channel CHANNEL] [--bitrate BITRATE] [--auto-match AUTO_MATCH] [--no-prompt]
[--list-ports] [--no-can] [--rio RIO] [--no-nt] [--timeout TIMEOUT]
[--publish-period PUBLISH_PERIOD] [--ui] [--ui-tcp-port UI_TCP_PORT] [--ui-only]
[--publish-can-summary] [--print-summary-period PRINT_SUMMARY_PERIOD]
[--console-monitor] [--console-transport {tcp,udp}] [--console-port CONSOLE_PORT]
[--console-host CONSOLE_HOST] [--console-rules CONSOLE_RULES]
[--console-timeout CONSOLE_TIMEOUT] [--console-rate CONSOLE_RATE]
[--console-debug-log CONSOLE_DEBUG_LOG] [--console-reset-on-start]
[--console-log-max-mb CONSOLE_LOG_MAX_MB] [--console-log-max-files CONSOLE_LOG_MAX_FILES]
[--startup-summary-after STARTUP_SUMMARY_AFTER] [--print-publish] [--stale-s STALE_S]
[--top-n TOP_N] [--dump-can-expected-ids DUMP_CAN_EXPECTED_IDS] [--dump-after DUMP_AFTER]
[--list-keys] [--dump-nt DUMP_NT] [--publish-unknown] [--dump-profile DUMP_PROFILE]
[--dump-profile-name DUMP_PROFILE_NAME] [--dump-profile-after DUMP_PROFILE_AFTER]
[--dump-profile-include-unknown] [--dump-api-inventory DUMP_API_INVENTORY]
[--dump-api-inventory-after DUMP_API_INVENTORY_AFTER] [--dump-can-config DUMP_CAN_CONFIG]
[--diff-inventory A.json B.json] [--diff-top DIFF_TOP] [--pcap PCAP] [--pcap-pipe PCAP_PIPE]
[--marker-id MARKER_ID] [--enable-markers] [--disable-markers] [--capture-note CAPTURE_NOTE]
[--tx-seq TX_SEQ] [--tx-allow] [--tx-scale TX_SCALE] [--tx-loop] [--tx-verbose]
[--print-status] [--print-control] [--print-any] [--print-can-id PRINT_CAN_ID]
[--print-device-id PRINT_DEVICE_ID] [--print-mfg PRINT_MFG] [--print-type PRINT_TYPE]
```

Script:

```
python tools\can_nt\can_nt_bridge.py [same options as above]
```

DESCRIPTION

Purpose: Provide a PC-side, read-only view of the robot CAN bus for bringup and diagnostics.

Output: NetworkTables diagnostics, console summaries, and optional PCAP/PCAPNG for Wireshark.

How it works: Opens a CANable/slcant interface, classifies frames, aggregates rates, and publishes.

How to use: Run during robot tests to verify device presence, traffic rates, and suspected issues.

Future: Extended reverse-engineering outputs (inventory diffs, fingerprints, hypotheses).

Safety: Never transmits CAN frames unless --tx-* is explicitly enabled with --tx-allow.

WORKFLOWS

1) Sync profiles -> run sniffer -> capture PCAP:

What it does: Ensures the PC tool and robot share the same profile metadata, then records raw CAN frames.

Output: Updated deploy profiles + a PCAPNG file for Wireshark review.

Commands:

```
python -m tools.sync_profiles
```

```
python -m tools.can_nt.can_nt_bridge --profile robot --pcap tools\can_nt\logs\run.pcapng
```

2) Sniffer -> inventory JSON -> config snapshot:

What it does: Samples live CAN traffic and converts observed device IDs into a reusable config.

Output: Inventory JSON + generated config JSON for tracking device presence over time.

Commands:

```
python -m tools.can_nt.can_nt_bridge --dump-api-inventory tools\can_nt\inv.json
```

```
python -m tools.can_inventory.can_inventory --generate --input tools\can_nt\inv.json --output
tools\can_nt\config.json --profileName robot
```

3) Control experiment capture:

What it does: Run a specific motor action while recording, so diffs isolate command-like frames.

Output: Two PCAPs with different actions (e.g., idle vs spin).

Commands:

```
python -m tools.can_nt.can_nt_bridge --pcap tools\can_nt\logs\idle.pcapng
```

```
python -m tools.can_nt.can_nt_bridge --pcap tools\can_nt\logs\spin.pcapng
```

EXAMPLES

```
python -m tools.can_nt.can_nt_bridge --profile robot --publish-can-summary
```

```
python -m tools.can_nt.can_nt_bridge --list-ports
```

```
python -m tools.can_nt.can_nt_bridge --ui --no-can --rio 172.22.11.2
```

```
python tools\can_nt\can_nt_bridge.py --profile demo_club --pcap tools\can_nt\logs\demo.pcapng
```

```
python -m tools.can_nt.can_nt_bridge --dump-profile tools\can_nt\sniffer_profile.json
```

tools.can_inventory.can_inventory

NAME

can_inventory - Generate/validate inventory configs and compare snapshots.

SYNOPSIS

Module:

```
python -m tools.can_inventory.can_inventory --generate --input <inventory.json> --output
<config.json> --profileName <name> [--compare <inventory.json>] [--timing-tolerance-percent
<percent>]
python -m tools.can_inventory.can_inventory --validate --input <config.json>
[--timing-tolerance-percent <percent>] [--inventory <inventory.json>] [--update-hash]
python -m tools.can_inventory.can_inventory --interactive
python -m tools.can_inventory.can_inventory --help
```

Script:

```
python tools\can_inventory\can_inventory.py --generate --input <inventory.json> --output
<config.json> --profileName <name> [--compare <inventory.json>] [--timing-tolerance-percent
<percent>]
python tools\can_inventory\can_inventory.py --validate --input <config.json>
[--timing-tolerance-percent <percent>] [--inventory <inventory.json>] [--update-hash]
python tools\can_inventory\can_inventory.py --interactive
python tools\can_inventory\can_inventory.py --help
```

DESCRIPTION

Purpose: Turn observed CAN traffic into a stable inventory/config for later comparisons.

Output: JSON config files and printed diffs between inventory snapshots.

How it works: Parses inventory JSON, aggregates devices, and validates timing tolerances.

How to use: Capture inventory during tests, then generate configs and diff between runs.

Future: Richer statistics (top talkers, fingerprints) and tighter schema checks.

WORKFLOWS

1) Capture inventory from live sniffer:

What it does: Records seen device IDs/rates while the robot is running.

Output: Inventory JSON suitable for config generation or diffing.

Commands:

```
python -m tools.can_nt.can_nt_bridge --dump-api-inventory inv.json
python -m tools.can_inventory.can_inventory --generate --input inv.json --output cfg.json
--profileName robot
```

2) Compare two inventories:

What it does: Highlights new/missing device pairs or rate changes between runs.

Output: Printed diff summary for quick regression checks.

Commands:

```
python -m tools.can_inventory.can_inventory --generate --input inv_new.json --output
cfg_new.json --profileName robot --compare inv_old.json
```

3) Validate timing tolerance:

What it does: Checks whether observed frame rates fall within expected tolerance.

Output: Pass/fail report for timing consistency.

Commands:

```
python -m tools.can_inventory.can_inventory --validate --input cfg.json --inventory inv.json
--timing-tolerance-percent 10
```

EXAMPLES

```
python -m tools.can_inventory.can_inventory --generate --input inv.json --output cfg.json
--profileName robot
python -m tools.can_inventory.can_inventory --generate --input inv.json --output cfg.json
--profileName robot --compare prev.json
python -m tools.can_inventory.can_inventory --validate --input cfg.json --inventory inv.json
python tools\can_inventory\can_inventory.py --interactive
```

tools.can_analyze_windows.analyze_can_windows

NAME

analyze_can_windows - Summarize CAN logs in time windows.

SYNOPSIS

Module:

```
python -m tools.can_analyze_windows.analyze_can_windows [-h] [--window WINDOW] [--relative]
[--mfg MFG] [--dtype DTYPE] [--device-id DEVICE_ID] [--top TOP] <log>
```

Script:

```
python tools\can_analyze_windows\analyze_can_windows.py [-h] [--window WINDOW] [--relative]
[--mfg MFG] [--dtype DTYPE] [--device-id DEVICE_ID] [--top TOP] <log>
```

DESCRIPTION

Purpose: Slice CAN logs into windows to see which API pairs dominate each phase.

Output: Printed summaries of API class/index counts per window.

How it works: Decodes arbitration IDs and counts frames in each specified time range.

How to use: Compare log windows before/after a robot action to spot command-like frames.

Future: Export JSON summaries and integrate with inventory diff tooling.

WORKFLOWS

1) Capture PCAP -> export log -> analyze windows:

What it does: Records raw traffic, then analyzes specific time windows to identify changes.

Output: Windowed API class/index summaries to compare phases of a test.

Commands:

```
python -m tools.can_nt.can_nt_bridge --pcap tools\can_nt\logs\run.pcapng
python -m tools.can_analyze_windows.analyze_can_windows logs\run.log --window 0,5,boot
```

2) Compare two action phases:

What it does: Create two windows (idle vs action) to spot likely command frames.

Output: Two summaries that can be diffed by eye.

Commands:

```
python -m tools.can_analyze_windows.analyze_can_windows logs\run.log --window 0,5,idle
--window 5,5,spin
```

EXAMPLES

```
python -m tools.can_analyze_windows.analyze_can_windows logs\run.log --window 0,5,boot
python -m tools.can_analyze_windows.analyze_can_windows logs\run.blf --relative --window
2,3,spinup
python tools\can_analyze_windows\analyze_can_windows.py logs\run.log --mfg 4 --dtype 7 --top
10
```

tools.sync_profiles

NAME

sync_profiles - Sync canonical profiles into deploy.

SYNOPSIS

Module:

```
python -m tools.sync_profiles [-h] [--source SOURCE] [--dest DEST]
```

Script:

```
python tools\sync_profiles.py [-h] [--source SOURCE] [--dest DEST]
```

DESCRIPTION

Purpose: Keep deploy profiles aligned with canonical data and hash/version metadata.

Output: Updated data/bringup_profiles.json and deploy copy with data_version/data_hash.

How it works: Loads JSON, validates schema_version, stamps version, computes hash, writes both.

How to use: Run after editing profiles so robot and PC tools load consistent data.

Future: Optional validation and diff reporting before write.

WORKFLOWS

1) Edit profiles -> validate -> sync:

What it does: Ensures schema correctness before deploying to the robot.

Output: Validation report + synced deploy file.

Commands:

```
python -m tools.can_topology.validate_profiles --path data\bringup_profiles.json --verbose
```

```
python -m tools.sync_profiles
```

EXAMPLES

```
python -m tools.sync_profiles
```

```
python -m tools.sync_profiles --source data\bringup_profiles.json --dest
```

```
src\main\deploy\bringup_profiles.json
```

tools.visualize_profiles

NAME

visualize_profiles - Render profiles to HTML diagram.

SYNOPSIS

Module:

```
python -m tools.visualize_profiles [-h] [--input INPUT] [--output OUTPUT]
```

Script:

```
python tools\visualize_profiles.py [-h] [--input INPUT] [--output OUTPUT]
```

DESCRIPTION

Purpose: Visualize bringup profiles for quick sanity checks and review.

Output: Self-contained HTML report with diagrams and tables.

How it works: Reads bringup_profiles.json and renders per-profile diagrams.

How to use: Generate after profile edits to review CAN ID layout.

Future: Highlight conflicts and link to device metadata.

WORKFLOWS

1) Update profiles -> sync -> visualize:

What it does: Produces an HTML diagram so you can visually inspect IDs and labels.

Output: Updated deploy profile + HTML report.

Commands:

```
python -m tools.sync_profiles
```

```
python -m tools.visualize_profiles --output docs\profiles.html
```

EXAMPLES

```
python -m tools.visualize_profiles
```

```
python -m tools.visualize_profiles --input data\bringup_profiles.json --output docs\profiles.html
```


tools.can_topology.can_table_import

NAME

can_table_import - Convert a CAN ID table to profiles JSON.

SYNOPSIS

Module:

```
python -m tools.can_topology.can_table_import [-h] --profile PROFILE [--input INPUT]
[--output OUTPUT] [--warn-duplicates]
```

Script:

```
python tools\can_topology\can_table_import.py [-h] --profile PROFILE [--input INPUT]
[--output OUTPUT] [--warn-duplicates]
```

DESCRIPTION

Purpose: Turn a simple CAN table into a profile JSON seed.

Output: bringup_profiles.json-style payload (single profile).

How it works: Parses columns, normalizes labels, assigns categories, writes JSON.

How to use: Start from a spreadsheet or text table and import into the editor.

Future: Add richer device tagging and validation before export.

WORKFLOWS

1) Table -> profile -> open editor -> sync:

What it does: Seeds a profile from a text table, then refines it in the GUI.

Output: bringup_profiles.json updated and synced to deploy.

Commands:

```
python -m tools.can_topology.can_table_import --profile robot --input can_table.txt --output
profile.json
```

```
python -m tools.can_topology.can_top_editor
```

```
python -m tools.sync_profiles
```

EXAMPLES

```
python -m tools.can_topology.can_table_import --profile robot --input can_table.txt --output
profile.json
```

```
python tools\can_topology\can_table_import.py --profile practice --input can_table.txt
--warn-duplicates
```

tools.can_topology.validate_profiles

NAME

validate_profiles - Validate bringup_profiles.json compatibility.

SYNOPSIS

Module:

```
python -m tools.can_topology.validate_profiles [-h] [--path PATH] [--strict] [--verbose]
```

Script:

```
python tools\can_topology\validate_profiles.py [-h] [--path PATH] [--strict] [--verbose]
```

DESCRIPTION

Purpose: Ensure profiles are safe for robot and PC tooling to consume.

Output: Pass/fail/warn report; nonzero exit on errors when strict.

How it works: Validates schema_version, data_hash, duplicates, and constraints.

How to use: Run after profile edits or before syncing to deploy.

Future: Add schema evolution checks and migration hints.

WORKFLOWS

1) After edits -> validate -> sync:

What it does: Prevents invalid profiles from reaching the robot/PC tools.

Output: Validation report + synced deploy file when clean.

Commands:

```
python -m tools.can_topology.validate_profiles --path data\bringup_profiles.json --verbose
```

```
python -m tools.sync_profiles
```

EXAMPLES

```
python -m tools.can_topology.validate_profiles
```

```
python -m tools.can_topology.validate_profiles --path data\bringup_profiles.json --verbose
```

tools.md_to_docx.md_to_docx

NAME

md_to_docx - Convert markdown to DOCX with TOC.

SYNOPSIS

Module:

```
python -m tools.md_to_docx.md_to_docx [-h] --input INPUT [--output OUTPUT] [--title TITLE]
```

Script:

```
python tools\md_to_docx\md_to_docx.py [-h] --input INPUT [--output OUTPUT] [--title TITLE]
```

DESCRIPTION

Purpose: Produce printable DOCX documents from Markdown sources.

Output: DOCX file with title page, TOC, and structured headings.

How it works: Parses Markdown lines and maps headings/lists to DOCX elements.

How to use: Generate handouts or test procedures for printing.

Future: Improved Markdown coverage and style templates.

WORKFLOWS

1) Update docs -> convert to DOCX:

What it does: Generates a printable DOCX with TOC from Markdown.

Output: DOCX ready for printing or sharing.

Commands:

```
python -m tools.md_to_docx.md_to_docx --input README.md --output docs\README.docx
```

EXAMPLES

```
python -m tools.md_to_docx.md_to_docx --input README.md
```

```
python -m tools.md_to_docx.md_to_docx --input README.md --output docs\README.docx --title "Bringup"
```

tools.add_journal_note

NAME

add_journal_note - Append a dated journal note.

SYNOPSIS

Module:

```
python -m tools.add_journal_note [-h] --text TEXT [--title TITLE] [--date DATE]
```

Script:

```
python tools\add_journal_note.py [-h] --text TEXT [--title TITLE] [--date DATE]
```

DESCRIPTION

Purpose: Capture a short dated log entry without manual file setup.

Output: A new Markdown note under notes/journal.

How it works: Builds a filename from date/title, writes the text to disk.

How to use: Record bringup observations or test results quickly.

Future: Attach tags or links to logs/captures.

WORKFLOWS

1) After test -> log note:

What it does: Captures a short observation with date metadata.

Output: Markdown note in notes/journal.

Commands:

```
python -m tools.add_journal_note --text "Observed dropouts on CAN"
```

EXAMPLES

```
python -m tools.add_journal_note --text "Ran bringup on practice bot"
```

```
python -m tools.add_journal_note --text "CANable test" --title "Bench" --date 2026-03-19
```

tools.add_tbd_note

NAME

add_tbd_note - Append a note to TBD.md.

SYNOPSIS

Module:

```
python -m tools.add_tbd_note [-h] --text TEXT [--path PATH] [--section SECTION]
```

Script:

```
python tools\add_tbd_note.py [-h] --text TEXT [--path PATH] [--section SECTION]
```

DESCRIPTION

Purpose: Keep a running task list without manually editing files.

Output: A new bullet under the chosen section in TBD.md.

How it works: Finds/creates the section header and appends a formatted entry.

How to use: Capture TODOs during bringup sessions.

Future: Optional priority tags and sorting.

WORKFLOWS

1) Add TODO after a session:

What it does: Adds a task entry to the standard TODO list.

Output: Updated TBD.md under the chosen section.

Commands:

```
python -m tools.add_tbd_note --text "Verify PDH current readings"
```

EXAMPLES

```
python -m tools.add_tbd_note --text "Verify CAN IDs on new module"
```

```
python -m tools.add_tbd_note --text "Update profiles" --section Planning
```

tools.bringup_test_wizard.gen_bringup_tests

NAME

gen_bringup_tests - Interactive bringup test generator.

SYNOPSIS

Module:

```
python -m tools.bringup_test_wizard.gen_bringup_tests
```

Script:

```
python tools\bringup_test_wizard\gen_bringup_tests.py
```

DESCRIPTION

Purpose: Build bringup_tests.json through guided prompts.

Output: Updated bringup_tests.json in deploy.

How it works: Asks for test types and parameters, then writes JSON.

How to use: Create or update tests without hand-editing JSON.

Future: Template presets and validation against profiles.

WORKFLOWS

1) Generate tests -> run UI:

What it does: Creates test definitions then uses the UI to trigger them on the robot.

Output: bringup_tests.json + UI ready for command dispatch.

Commands:

```
python -m tools.bringup_test_wizard.gen_bringup_tests
```

```
python -m tools.can_nt.can_nt_bridge --ui --no-can
```

EXAMPLES

```
python -m tools.bringup_test_wizard.gen_bringup_tests
```

tools.test_template_wizard.copy_test_template

NAME

copy_test_template - Interactive test template copier.

SYNOPSIS

Module:

```
python -m tools.test_template_wizard.copy_test_template
```

Script:

```
python tools\test_template_wizard\copy_test_template.py
```

DESCRIPTION

Purpose: Start from a known template and customize quickly.

Output: Updated bringup_tests.json in deploy.

How it works: Copies a template, prompts for edits, writes JSON.

How to use: Use when you want a consistent test structure.

Future: More templates and validation hooks.

WORKFLOWS

1) Copy template -> run tests UI:

What it does: Uses a template to quickly populate bringup_tests.json.

Output: Updated bringup_tests.json + UI ready for dispatch.

Commands:

```
python -m tools.test_template_wizard.copy_test_template
```

```
python -m tools.can_nt.can_nt_bridge --ui --no-can
```

EXAMPLES

```
python -m tools.test_template_wizard.copy_test_template
```

tools.can_topology.can_top_editor

NAME

can_top_editor - GUI topology editor.

SYNOPSIS

Module:

```
python -m tools.can_topology.can_top_editor
```

Script:

```
python tools\can_topology\can_top_editor.py
```

DESCRIPTION

Purpose: Visually author CAN topology and profile data.

Output: bringup_profiles.json with optional diagram metadata.

How it works: GUI canvas for nodes, categories, and export actions.

How to use: Create or modify profiles without hand-editing JSON.

Future: Built-in diff/merge and validation warnings.

WORKFLOWS

1) Edit topology -> sync -> visualize:

What it does: Creates/updates profiles, syncs deploy, and renders a visual check.

Output: Updated bringup_profiles.json, deploy copy, and HTML diagram.

Commands:

```
python -m tools.can_topology.can_top_editor
```

```
python -m tools.sync_profiles
```

```
python -m tools.visualize_profiles
```

EXAMPLES

```
python -m tools.can_topology.can_top_editor
```


tools.can_topology.legacy.can_topology_editor_OLD

NAME

can_topology_editor_OLD - Legacy GUI editor (deprecated).

SYNOPSIS

Module:

```
python -m tools.can_topology.legacy.can_topology_editor_OLD
```

Script:

```
python tools\can_topology\legacy\can_topology_editor_OLD.py
```

DESCRIPTION

Purpose: Legacy editor retained for reference only.

Output: Older style profile exports.

How it works: Older GUI implementation with separate logic.

How to use: Avoid for new work; keep only for reference.

Future: Removal once legacy use ends.

WORKFLOWS

1) Legacy review only:

What it does: Opens the old GUI for reference only.

Output: Legacy file edits if you choose to save.

Commands:

```
python -m tools.can_topology.legacy.can_topology_editor_OLD
```

EXAMPLES

```
python -m tools.can_topology.legacy.can_topology_editor_OLD
```

tools.gen_cli_cheatsheet_pdf

NAME

gen_cli_cheatsheet_pdf - Generate the printable PDF cheat sheet.

SYNOPSIS

Module:

```
python -m tools.gen_cli_cheatsheet_pdf [--output PATH]
```

Script:

```
python tools\gen_cli_cheatsheet_pdf.py [--output PATH]
```

DESCRIPTION

Purpose: Produce a printable reference for all tool CLIs.

Output: PDF in docs/ by default.

How it works: Renders manpage-style sections into a PDF.

How to use: Run after tool updates or before printing.

Future: Optional Markdown/HTML outputs.

WORKFLOWS

1) Update docs -> regenerate PDF:

What it does: Regenerates the printable cheat sheet with latest commands.

Output: Updated PDF in docs/.

Commands:

```
python -m tools.gen_cli_cheatsheet_pdf
```

EXAMPLES

```
python -m tools.gen_cli_cheatsheet_pdf
```

```
python -m tools.gen_cli_cheatsheet_pdf --output docs\cli_cheatsheet.pdf
```